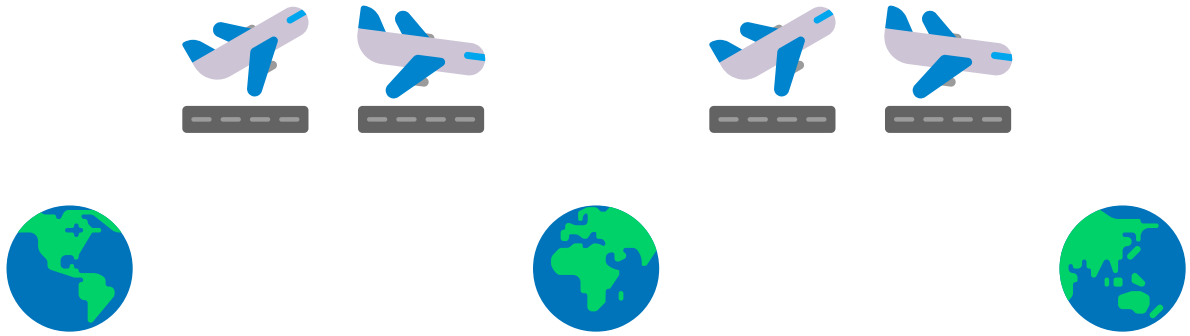
Web Engineering WEE (Z-TIA-23-T-a)

Case Study

Flugbuchungssystem mit einer Single Page Application, einem RESTful Web Service und einer relationalen Datenbank



Ausgangslage

User Story

Als Benutzer Felix Huber möchte ich Flüge suchen, filtern und buchen, damit ich meine Reisen einfach und effizient planen kann.

Beschreibung

Felix Huber benötigt eine Webapplikation, die ihm erlaubt,

- eine Übersicht über verfügbare Flüge zu erhalten
- Flüge nach Kriterien wie Abreiseort, Zielort, Datum, Zeit, Fluggesellschaft, Preis und Ticketverfügbarkeit zu filtern
- direkt eine Buchung durchzuführen
- eine Übersicht über seine bisherigen Buchungen (d.i. eine historische Ansicht) einzusehen

Akzeptanzkriterien

Felix kann Flüge nach Abreiseort, Zielort, Datum, Zeit, Fluggesellschaft, Preis und Ticketverfügbarkeit filtern.

- Die Ergebnisse werden in einer übersichtlichen Liste angezeigt.
- Felix kann einen Flug auswählen und direkt buchen.
- Nach erfolgreicher Buchung wird diese in seiner persönlichen Buchungshistorie gespeichert.
- Felix kann jederzeit seine Buchungshistorie einsehen.
- Das System speichert alle Daten persistent in einer Datenbank.
- Die Oberfläche ist responsiv und intuitiv bedienbar.

Technische Anforderungen

Das Client Tier ist als Single Page Application (SPA) unter Verwendung von HTML5 / CSS3 / Bootstrap 5 / JavaScript oder React, das Server Tier als Middleware unter Verwendung von Jakarta EE (insbesondere JAX-RS und JPA) oder Node.js /Express oder PHP und das EIS Tier (d.h. die Persistenz) vorzugsweise mit einer MySQL-Datenbank umzusetzen.

Jegliche Abweichung bedingt einen zeitnahen Antrag mit Begründung in schriftlicher Form und eine darauffolgende Bewilligung.

Aufgabenstellung (Epics & Stories)

Epic 1: Flugübersicht & Filter

User Story

Als Felix möchte ich Flüge nach Kriterien filtern, um passende Angebote schnell zu finden.

Tasks

- Frontend:
  UI für Suchmaske mit Eingabefeldern (Abreiseort, Zielort, Datum, Zeit, Fluggesellschaft, Preis und Ticketverfügbarkeit)
- Backend:
  REST-Endpunkt `/flights` mit Filterparametern
- Datenbank:
  Tabellenstruktur für Flüge (inkl. Fluggesellschaft, Preis und Ticketverfügbarkeit)
- Integration:
  API-Call vom Frontend ans Backend mit Filterung
- Testing:
  Testprotokoll der UI-Tests für Suchmaske

Epic 2: Buchungssystem

User Story

Als Felix möchte ich einen Flug direkt buchen, damit ich meine Reise sichern kann.

Tasks

- Frontend:
  "Buchen"-Button und Checkout-Flow (Formular für Zahlungsdetails)
- Backend:
  REST-Endpunkt `/bookings` für neue Buchungen
- Datenbank:
  Tabelle für Buchungen mit Referenz zu Flug und User
- Validierung:
  Verfügbare Tickets prüfen vor Kaufabschluss
- Testing:
  Testprotokoll der End-to-End-Test für Buchungsprozess

Epic 3: Buchungshistorie

User Story

Als Felix möchte ich meine bisherigen Buchungen sehen, um meine Reisen im Überblick zu behalten.

Tasks

- Frontend:
  Seite "Meine Buchungen" mit Liste vergangener Flüge
- Backend:
  REST-Endpunkt `/bookings/{userId}` für historische Daten
- Datenbank:
  Query für Buchungshistorie
- Integration:
  Sortierung nach Datum, Filteroptionen (z. B. Fluggesellschaft, Preis)
- Testing:
  Testprotokoll der API-Tests für Buchungshistorie, UI-Tests für Darstellung

Epic 4: Technische Basis

User Story

Als Entwickler möchte ich eine stabile Architektur, damit die App wartbar und skalierbar ist.

Tasks

- Setup Frontend-Projekt:
  HTML5, CSS3, Bootstrap 5, JavaScript *oder*
  React
- Setup Backend-Projekt:
  Jakarta EE (insbesondere JAX-RS und JPA) *oder*
  Node.js /Express *oder*
  PHP
- Setup Datenbank:
  MySQL
- Dokumentation
  Installationsanleitung: Datei namens `README.md` (z.B. zu verwendende Software inkl. Versionen, Skript zum Anlegen der Datenbank, allfällige Logins) *und*
  sämtliche Testprotokolle (gem. Epics) *und*
  Reflexion (ca. eine halbe A4-Seite)

Epic 5: Nicht-funktionale Anforderungen

User Story

Als Felix möchte ich eine schnelle und sichere App, damit ich Vertrauen haben.

Tasks

- Responsives Design für Desktop, allenfalls Mobile
- Authentifizierung & Autorisierung (Login, Session-Handling)
- Performance-Optimierung (Caching, Pagination)
- Logging
- Security (sichere Passwortspeicherung, Schutz vor SQL-Injection)

Abgabe

Die Case Study muss mit allen geforderten Dokumenten als Datei namens `CS_WEE_2025.zip` in Deinem Abgabeordner auf Teams eingereicht werden.

Zeitpunkt

Die Einreichung der Case Study muss bis SO, 22.03.2026, 23:59 erfolgen; eine verspätete Abgabe führt zu einem Abzug von 1.0 bei der finalen Bewertung.